Accelerating Super-Resolution CNN Inference via GPU Spatial Reduction: A Cross-Platform Study on Unified and Discrete Memory Architectures

M. Hamdani Ilham Latjoro, Adnan
Department of Informatics Engineering, Faculty of Engineering
Hasanuddin University
Makassar, Indonesia
latjoromhi25d@student.unhas.ac.id, adnan@unhas.ac.id

***Abstract*—Convolutional Neural Network (CNN) super-resolution models such as FSRCNN suffer from a severe computational bottleneck in their final deconvolution layer, where naive multi-threaded CPU accumulation causes data races and non-reproducible output corruption. We propose a GPU Spatial Reduction strategy that offloads FSRCNN's Layer 8 deconvolution to isolated per-channel CUDA kernels followed by a parallel reduction pass, eliminating races by construction, and evaluate it on two architecturally distinct platforms: the NVIDIA GB10 Grace Blackwell Superchip (unified memory over NVLink-C2C) and a discrete-GPU desktop (Intel i9-14900K with an NVIDIA RTX 4090 over PCIe). On both platforms, V1 and V2 output is byte-for-byte identical to a verified deterministic reference at every configuration tested, and the GPU output is further bit-identical across the two platforms directly (0 differing bytes, matching SHA-256 digests) despite differing host ISA and GPU microarchitecture. Timings are reported as mean ± SD over the 6 measured runs of a 7-run repetition. The warp-aligned 32×8 CUDA grid is fastest at the kernel level on both GPU architectures, and per-kernel event profiling localizes the entire gain to the deconvolution kernel, exactly where the coalescing explanation predicts it. Against each platform's best correctness-verified CPU configuration, GPU offloading achieves $1.46\times$ (GB10, 31.5 FPS) and up to $1.63\times$ (RTX 4090, 27–27.6 FPS) speedups; against the fastest bit-exact serial run the secondary figure is $3.84\times$ on the GB10. The determinism guarantee costs 4.6% of GPU Layer 8 time on the GB10 but only 0.6% on the RTX 4090, a gap traceable to whether the private buffer fits in last-level cache; conversely, device-to-host transfer is $4.74\times$ faster on unified memory. Against a constructed high-resolution reference the pipeline gains 2.17 dB PSNR-Y over bicubic upscaling. These results establish spatial reduction as a deterministic, cross-platform-portable pattern for GPU-accelerated CNN inference.**

***Index Terms*—Super-resolution, FSRCNN, GPU offloading, CUDA, spatial reduction, Grace Blackwell, asymmetric multi-core, bit-exactness.**

I. Introduction

Edge and workstation-class AI hardware increasingly adopt SoCs where multicore CPUs and GPUs co-exist on high-speed interconnects [3], [4]. Super-resolution CNNs progressed from early formulations [11] to FSRCNN [2], which defers feature extraction to low-resolution space and upscales only at the final deconvolution layer.

That final stage (Layer 8) carries a disproportionate cost: for $2\times$ upscaling it applies a 9×9 kernel across 56 channels and accumulates them into one output plane. Parallelizing it on a multicore CPU poses a dilemma — naive shared accumulation races and corrupts output non-reproducibly [1], while mutexes or atomics bring prohibitive contention. Privatize-then-reduce avoids both by writing per-channel private buffers and reducing them in a separate deterministic pass, at the price of 43.3 MiB of extra traffic.

We offload that deconvolution and reduction to CUDA while parallelizing Layers 1–7 across CPU cores. Our contributions:

1) *GPU cost-profile shift.* Privatize-then-reduce is a classical pattern, not a new algorithm; we show that moving it to the GPU changes its cost qualitatively. The private-buffer traffic that costs 8–18% of CPU wall time falls to 4.6% (GB10) and 0.6% (RTX 4090) of GPU Layer 8 time while output stays bit-exact. We frame this as bandwidth and parallelism rather than capacity: our primary platform shares one 128 GB pool between CPU and GPU, so the buffer does not move, only the computation over it does.
2) *Warp-aligned grid configuration.* A 32×8 thread block maximizes coalescing for the deconvolution's row-major access, cutting Layer 8 kernel time 23.4% (GB10) and 17.4% (RTX 4090); per-kernel profiling localizes the entire gain to the deconvolution, as the coalescing explanation predicts.
3) *Cross-platform determinism.* GPU output is byte-identical to the CPU baseline at every configuration tested on both platforms, and bit-identical *across* them (matching SHA-256), spanning two host ISAs and two GPU generations. Speedups over each platform's best deterministic CPU configuration are $1.46\times$ and $1.63\times$.
4) *An architectural boundary for the pattern's cost.* Per-kernel profiling shows the reduction is bandwidth-bound and its cost depends on whether the private buffer fits last-level cache: near-free on the RTX 4090, whose L2 holds it, and $10\times$ costlier on the GB10, which streams it at 95% of memory peak. Device-to-host transfer runs

the opposite way, $4.74\times$ faster on unified memory.

II. Background and Baseline Architecture

A. Related Work

CNN super-resolution progressed from early formulations [11] to FSRCNN [2], increasingly deployed on heterogeneous CPU+GPU SoCs [3], [4]. Closest to ours, Annisa et al. [1] characterize the same FSRCNN Layer 8 race on an Orange Pi 5 (Rockchip RK3588S, same Suzie sequence), finding corruption more severe under LITTLE-core affinity; their focus is characterizing that corruption across affinity settings rather than remedying it algorithmically. We do not compare speedups directly, our hardware differing from theirs by orders of magnitude in core count and class. Privatize-then-reduce is a classical pattern; our contribution is adopting it as a race-free CPU baseline and then moving it to the GPU, which changes its cost profile while preserving bit-exactness. CNN inference scales well on ARM big.LITTLE clusters under proper scheduling [5], supporting our GB10 CPU baseline as non-trivial. General-purpose libraries such as cuDNN [15] target convolution broadly but not this race-free layout (Section III-B); we do not claim to outperform them, only to control the memory layout. Discrete-GPU PCIe overhead is well documented [9], motivating our unified-versus-discrete comparison, and recent work characterizes unified physical memory for accelerators directly [21]. Our RTX 4090 scheduling collapse relates to prior work on asymmetric-CPU scheduling [6], [7]; there our contribution is an empirical observation, not a mechanism.

B. FSRCNN Architecture

FSRCNN consists of eight layers: Feature Extraction (Layer 1), Shrinking (Layer 2), four Mapping layers (Layers 3–6), Expanding (Layer 7), and Deconvolution (Layer 8). For an input sequence at 176×144 scaled by $2\times$, Layers 1–7 operate in low-resolution (176×144) space, outputting 56 feature maps. Layer 8 receives these 56 channels, applies a 9×9 transposed convolution [16] with stride 2, and reduces them into a 352×288 YUV luminance output frame.

C. Baseline Variants

We compare three variants. **V0 (naive OpenMP CPU)** has Layer 8 worker threads accumulate deconvolution outputs directly into one shared buffer `img_fltr_8`; beyond one thread, overlapping kernel footprints cause unsynchronized read-modify-write races and output corruption — the class of bug dynamic detectors such as ThreadSanitizer [13] exist to catch. **V1 (spatial reduction, CPU)** eliminates the race with a private buffer all_tmp$[56\times352\times288]$ of doubles (43.3 MiB), each channel writing its own isolated offset, followed by a second pass summing the 56 channels per pixel. Both passes are parallel but along different axes: deconvolution over channels, reduction over output pixels. The reduction needs no synchronization because each output pixel is written by exactly one thread, and stays deterministic because that thread accumulates its 56 summands in fixed channel order regardless of scheduling. **V2 (GPU spatial reduction, proposed)** keeps Layers 1–7 on OpenMP CPU threads and offloads the Layer 8 deconvolution and reduction to CUDA.

III. GPU Spatial Reduction Implementation

A. Hardware Targets: GB10 and RTX 4090

Our primary platform is the ASUS Ascent GX10 with the NVIDIA GB10 Grace Blackwell Superchip [17], whose CPU cluster uses Arm's DynamIQ big.LITTLE technology [18]. Section VI cross-validates on an architecturally distinct second platform: an Intel Core i9-14900K desktop with an RTX 4090 (Ada Lovelace [19]), where CPU and GPU memory are physically separate and PCIe-connected rather than sharing the GB10's unified NVLink-C2C pool. Table I compares them.

TABLE I
Hardware Specifications: GB10 (GX10) vs. RTX 4090 Desktop

Component	GB10 (ASUS GX10)	RTX 4090 Desktop
CPU	20-core ARM v9.2-A DynamIQ 10× X925 P + 10× A725 E @ 3.90/2.81 GHz, no SMT	Intel Core i9-14900K 8 P-core (w/HT) + 16 E-core 24C/32T, up to 6.0 GHz
GPU	NVIDIA Blackwell, `sm_121`	NVIDIA Ada Lovelace, `sm_89`
GPU Cores	6,144 CUDA, 48 SMs, Gen5 Tensor	16,384 CUDA, 128 SMs [19]
Memory	128 GB LPDDR5x, *unified*	64 GB DDR (63 GB OS-visible) + 24 GB GDDR6X, *discrete*
Interconnect	NVLink-C2C (5× PCIe 5.0)	PCIe (CPU ↔ GPU)
CUDA	13.0, Driver 580.159.03, `V13.0.88`	13.0, Driver 580.65.06, `V13.0.48`
GCC	13.3.0 (Ubuntu 24.04)	13.3.0 (Ubuntu 24.04)
OS	Ubuntu 24.04 LTS (aarch64)	Ubuntu 24.04 LTS (x86_64)

B. CUDA Kernel Design and Warp Alignment

We implement Layer 8 as two hand-written CUDA kernels rather than using a general-purpose library such as cuDNN [15]. We do not claim this beats a cuDNN-based deconvolution — untested, and a well-tuned library kernel could plausibly win — only that it gives direct control over the private-buffer-plus-reduction layout central to our correctness argument (Section VII).

Deconvolution kernel. Each thread block processes a 2D tile of the input feature map. For each channel $c \in [0, 55]$, threads compute the 9×9 transposed convolution and write into an isolated channel plane of `d_all_tmp`, indexed $c \times (H_{out}W_{out}) + yW_{out} + x$.

Spatial reduction kernel. One thread per output pixel (y, x) iterates all 56 planes, sums them, and adds the Layer 8 bias:

$$\text{output}[y, x] = \left(\sum_{c=0}^{55} \text{d\_all\_tmp}[c, y, x]\right) + \text{bias}_8 \qquad (1)$$

Channel writes are isolated during deconvolution and read-only during reduction, so execution is race-free and deterministic by construction.

NVIDIA GPUs execute threads in SIMD groups of 32 called *warps* [8]. A standard 16×16 block places only 16 threads along the row-major X dimension, so consecutive warps straddle non-contiguous rows. Since warp and block sizing has a first-order effect on SIMT throughput [22], we evaluate a 32×8 block (256 threads): setting $B_x = 32$ guarantees all 32 threads of a warp access contiguous doubles along one image row, maximizing coalescing.

IV. Experimental Evaluation

A. Evaluation Methodology

The workload is 150 frames of the Suzie sequence (176×144 → 352×288, YUV420p). Every configuration ran 7 times: run 1 discarded as warmup, runs 2–7 recorded as mean ± SD.

Reproducibility: CPU binaries use `gcc -fopenmp -O3` (no `-march=native` or `-ffast-math`), with plain `#pragma omp parallel for` and no explicit `schedule()`, so placement follows `libgomp`'s static default. The GPU binary uses `nvcc -O3 -std=c++11 -arch=native`, resolving to `sm_121` and `sm_89` respectively. Both platforms run GCC 13.3.0 and CUDA 13.0, differing only in CUDA patch level (Table I). All arithmetic is double precision, matching the CPU baseline so the bit-exact CPU≡GPU comparison is as strict as possible; FP32 would admit rounding divergence even for a logically equivalent computation (throughput consequence in Section VI).

Reference file: We generate one deterministic reference by running V1 single-threaded once. Since V0, V1 and V2 implement the identical algorithm and differ only in execution strategy, this is an internal determinism check, not an independently sourced high-resolution ground truth; Section IV-C addresses reconstruction quality separately. Correctness is assessed by direct byte comparison (`cmp -l`, diff_bytes = 0 for V1 against V2 across all grid configurations) and by PSNR/SSIM [10] via `ffmpeg`. V1/V2's PSNR= ∞ against this file is logically equivalent to the byte comparison rather than a separate quality measurement, so we report it as *bit-exact*.

PSNR/SSIM in Table II are `ffmpeg`'s all-component averages. Because only the Y plane passes through the network, untouched chroma pulls them upward — the RTX 4090's worst V0 case reads 23.0 dB/0.845 all-component but 21.5 dB/0.767 on luminance alone — so these figures *understate* the corruption. We give luminance-only numbers wherever its magnitude matters.

V0 degrades non-monotonically from its data race: 13.8 dB/0.518 on the GB10 (4 threads), and 68.6 dB/0.99996 on the RTX 4090 (4 threads) before collapsing to 23.0 dB/0.845 at 32 threads. The RTX case is the instructive one — a race costing only 68.6 dB is invisible to casual inspection or a coarse PSNR threshold, yet is exactly as non-deterministic as the 13.8 dB case, which argues for determinism by construction over trusting that runs "look fine." Three consecutive V0 runs at 32 threads corrupted 4.75, 5.01 and 5.04 million of 22.8 million output bytes (21.34, 21.46, 21.63 dB PSNR-Y): never the same output twice. Fig. 1 shows this directly — reference, V1 and V2 are visually identical, while V0 carries clear scanline corruption from unsynchronized writes to `img_fltr_8`.

B. Performance Scaling & Accuracy Benchmark

Table II presents results for both platforms across thread counts, accuracy, and GPU grid configurations.

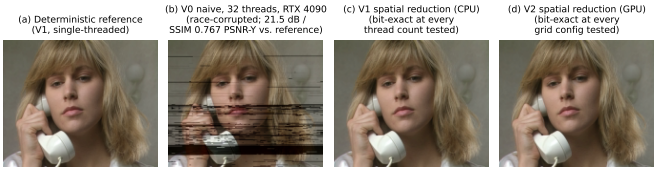

Fig. 1. Frame 26 of the reconstructed Suzie sequence across all four output variants. (a) Verified deterministic reference (V1, single-threaded). (b) V0's naive accumulation at 32 threads on the RTX 4090 (Intel i9-14900K host), visibly corrupted by unsynchronized concurrent writes to `img_fltr_8` (horizontal scanline artifacts); this is one representative run of three, which scored 21.34–21.63 dB PSNR-Y and 0.767–0.779 SSIM-Y against (a) and never reproduced the same output twice. (c) V1 (CPU spatial reduction) and (d) V2 (GPU spatial reduction) are both pixel-identical to (a) at every thread count and grid configuration tested, visually indistinguishable from the reference unlike (b).

C. Reconstruction Quality Against a High-Resolution Reference

Every correctness figure so far is a self-consistency check: V2 against V1, which is why it reads "bit-exact." That establishes the GPU port is faithful to the CPU algorithm, but says nothing about whether FSRCNN reconstructs anything, since no high-resolution ground truth exists in the pipeline — Suzie is the input, not a downsampled version of something larger.

We therefore constructed the missing reference: Suzie was bicubic-downsampled to 88×72, fed to the same pipeline at scale 2 to reconstruct 176×144, and scored against the original. Bicubic upscaling of that same input is the comparator — what the network must beat to justify its runtime. We report luminance only, since only Y passes through the network and an all-component average would credit FSRCNN for a chroma replication step it does not perform.

FSRCNN reaches 37.86 dB PSNR-Y and 0.9665 SSIM-Y against bicubic's 35.69 dB and 0.9532: +2.17 dB and +0.0133 over 150 frames. The network earns its cost. Two caveats belong with that: it is one sequence at one scale factor, a sanity check on the pipeline rather than a claim about FSRCNN in general [2]; and on chroma bicubic is in fact *better* (50.45/49.71 against 48.56/47.48 dB), precisely because it interpolates U and V where our pipeline replicates them — a limitation of this implementation, not of spatial reduction.

D. The Cost of Determinism, Quantified

Contribution 1 (Section I) claims that privatization-reduction is expensive on the CPU and absorbed by the GPU. Table II lets us quantify the CPU side directly as the wall-time delta between V0 (racy, no privatization) and V1 (privatized, deterministic) at matched thread counts. On the GB10: +18.2% at 1 thread (18,240.50 → 21,552.33 ms), +17.6% at 4 threads (7,074.33 → 8,316.50 ms), +9.1% at 20 threads (6,346.33 → 6,921.83 ms). On the RTX 4090: +9.2% at 1 thread (20,973.50 → 22,904.17 ms), +7.8% at 4 threads (8,207.67 → 8,844.17 ms). This delta isolates privatization specifically rather than a difference in how much of the work is threaded: V0 and V1 parallelize the same deconvolution, and V1's additional reduction pass is itself parallel over output

TABLE II
KEY CONFIGURATIONS ON BOTH PLATFORMS: PERFORMANCE AND DEVIATION FROM EACH PLATFORM'S VERIFIED DETERMINISTIC REFERENCE. WALL TIMES ARE MEAN ± SD OVER THE 6 MEASURED RUNS OF A 7-RUN REPETITION.

Variant	Configuration / Threads	Wall Time (ms)	Speedup vs. Serial	Speedup vs. Best CPU	Dev. from Ref. (dB)	SSIM	GPU L8 Kernel (ms)
GB10 (ASUS GX10), unified memory — CPU thread ladder 1–20							
CPU V0 (Naive)	1 Thread	$18{,}240.50 \pm 228.82$	1.00×	N/A	bit-exact	bit-exact	N/A
CPU V0 (Naive)	4 Threads (worst)	$7{,}074.33 \pm 90.21$	2.58×	N/A	13.785	0.518246	N/A
CPU V0 (Naive)	20 Threads (Max, fastest)	$6{,}346.33 \pm 32.60$	2.87×	N/A	25.751	0.838930	N/A
CPU V1 (Baseline)	1 Thread (Serial)	$21{,}552.33 \pm 298.07$	1.00×	N/A	bit-exact	bit-exact	N/A
CPU V1	4 Threads (cf. RTX optimum)	$8{,}316.50 \pm 58.06$	2.59×	N/A	bit-exact	bit-exact	N/A
CPU V1 (Max Cores)	20 Threads (fastest verified)	$6{,}921.83 \pm 82.49$	3.11×	N/A	bit-exact	bit-exact	N/A
GPU V2 (Hybrid)	Grid 16×16_256 (baseline)	$4{,}935.67 \pm 36.30$	4.37×	1.40×	bit-exact	bit-exact	696.51 ± 1.78
GPU V2 (Best Grid)	Grid 32×8_256	$4{,}756.33 \pm 30.71$	4.53×	1.46×	bit-exact	bit-exact	533.19 ± 2.58
RTX 4090 desktop, discrete memory — CPU thread ladder 1–32, 4-thread GPU backdrop							
CPU V0 (Naive)	1 Thread	$20{,}973.50 \pm 36.30$	1.00×	N/A	bit-exact	bit-exact	N/A
CPU V0 (Naive)	4 Threads (fastest)	$8{,}207.67 \pm 247.16$	2.56×	N/A	68.559	0.999957	N/A
CPU V0 (Naive)	32 Threads (Max, worst)	$26{,}388.67 \pm 664.00$	0.79×	N/A	23.006	0.845283	N/A
CPU V1 (Baseline)	1 Thread (Serial)	$22{,}904.17 \pm 86.90$	1.00×	N/A	bit-exact	bit-exact	N/A
CPU V1 (Best)	4 Threads (fastest verified)	$8{,}844.17 \pm 132.68$	2.59×	N/A	bit-exact	bit-exact	N/A
GPU V2 (Hybrid)	Grid 16×16_256 (baseline)	$5{,}579.00 \pm 166.61$	4.11×	1.59×	bit-exact	bit-exact	437.06 ± 3.86
GPU V2 (Best Grid)	Grid 32×8_256	$5{,}435.50 \pm 148.01$	4.21×	1.63×	bit-exact	bit-exact	360.92 ± 7.22

Speedup vs. Serial: CPU rows against their own variant's 1-thread time; GPU V2 has none of its own, so its denominator is CPU V1's serial time on that platform (Section V-A also reports it against the faster bit-exact V0 serial time). *Speedup vs. Best CPU*: GPU V2 only, against CPU V1's fastest correctness-verified configuration. *Dev./SSIM* are against each platform's own deterministic reference; "bit-exact" is logically equivalent to `cmp -l`, not an independent quality measurement, and figures are all-component averages (Section IV-A). *GPU L8 Kernel*: mean ± SD over 6 runs of the profiling sweep (Section IV-E); remaining grid configs in Fig. 2.

pixels (Section II-C), so the gap measures the cost of writing and re-reading 43.3 MiB of private buffers, not the cost of serializing a stage. Determinism therefore costs 8–18% of CPU wall time. The GB10 points suggest the cost falls once the machine is saturated, but 1 thread and 4 threads are nearly identical there (18.2% vs. 17.6%), so this is a two-level effect at best, not a smooth trend.

On the GPU side, per-kernel instrumentation (Section IV-E) puts the same cost in comparable units. The reduction pass is the GPU analogue of the privatization overhead, and it accounts for 4.6% of the Layer 8 event window on the GB10 and 0.6% on the RTX 4090 — in both cases well below the CPU's 8–18%, and on the discrete GPU almost free. Peak device memory stays at 44.3 MiB including allocator padding, under 0.1% of either platform's pool; we call this a bandwidth-and-parallelism result rather than a capacity one because on the GB10 there is no separate VRAM for the buffer to move into.

The gap between the two GPUs is architectural rather than algorithmic: the reduction is bandwidth-bound, and the RTX 4090's L2 is large enough to hold the entire private buffer (Section IV-E). Determinism by construction is therefore cheapest exactly where cache capacity exceeds the privatization working set — a useful design rule for applying this pattern to a larger network, since the overhead scales with how far the private buffer overflows last-level cache rather than with channel count as such.

E. Fine-Grained GPU Timing Breakdown

We instrumented CUDA Event profiling inside the GPU binary and ran the optimal configuration (32×8_256) six times on each platform (Table III).

A correction to our earlier reading of this instrumentation. An earlier draft treated the host-side counter `h2d_ms` as a disjoint slice of wall time and reported host-to-device transfer as 516.74 ms, roughly 11% of the pipeline. That was wrong. The counter accumulates host time spent inside `cudaMemcpy`, but those copies are issued *inside* the event window `gpu_l8_ms` measures, and a blocking copy on the default stream cannot begin until the previously launched kernel has drained; `h2d_ms` therefore charges kernel-completion waiting to transfer and double-counts time already inside `gpu_l8_ms`. The six-run sweep makes this visible: `h2d_ms` tracks `gpu_l8_ms` at a nearly fixed ratio across all five grid configurations (0.96–0.98 on the GB10, 0.88–0.90 on the RTX 4090), and moves by 167 ms on the GB10 when only the *kernel block shape* changes, though the transferred byte count is identical in every configuration. No genuine transfer measurement behaves that way. We retract the "H2D asymmetry" claim that rested on it (Section V-B) and report the corrected decomposition below, obtained by bracketing each `deconv_kernel` launch and the `spatial_reduction_kernel` with their own events. Residual in-window time — real transfer, launch gaps, and host-side padding — is reported as its own line rather than folded into either kernel.

Two results stand out, and they point in opposite directions.

First, the reduction kernel costs 26.32 ms on the GB10 but only 2.62 ms on the RTX 4090, a 10.0× gap far larger than any general compute advantage (`deconv_kernel` differs by only 1.6×). The reduction is purely bandwidth-bound, reading the whole 43.31 MiB private buffer once per frame, 6.81 GB over 150 frames. That is 259 GB/s on the GB10, about 95% of the 273 GB/s peak of its LPDDR5x pool, and 2,597 GB/s on the RTX 4090 — more than 2.5× that card's 1,008 GB/s of GDDR6X bandwidth. A DRAM-resident working set cannot produce that number. The natural explanation is the RTX 4090's 72 MB L2: the buffer fits inside it, so the reduction is

TABLE III
FINE-GRAINED PROFILING BREAKDOWN, OPTIMAL CONFIGURATION (32×8_256, 150 FRAMES), MEAN ± SD OVER 6 RUNS

	GB10 (unified)		RTX 4090 (discrete)	
Pipeline Stage	Duration (ms)	%	Duration (ms)	%
CPU Layers 1–7 (`cpu_l17_ms`)	$3{,}392.82 \pm 21.62$	71.4%	$4{,}624.50 \pm 70.96$	84.5%
GPU Layer 8, event window (`gpu_l8_ms`)	533.19 ± 2.58	11.2%	360.92 ± 7.22	6.6%
↪ `deconv_kernel` (56×)	424.07 ± 0.76	73.4%†	261.90 ± 3.09	62.9%†
↪ `spatial_reduction_kernel` (1×)	26.32 ± 1.02	4.6%†	2.62 ± 0.01	0.6%†
↪ in-window transfer and launch gaps	127.65 ± 4.10	22.1%†	152.14 ± 0.36	36.5%†
Device-to-Host (`d2h_ms`)	3.46 ± 0.06	0.1%	16.42 ± 0.49	0.3%
Unaccounted (process launch, file I/O)	821.20	17.3%	469.33	8.6%
Total Pipeline Wall Time	$4{,}750.67 \pm 30.40$	100.0%	$5{,}471.17 \pm 76.17$	100.0%

†Indented rows are a decomposition of the `gpu_l8_ms` window directly above them, and their percentages are shares *of that window*, not of wall time; they are not additional wall-time slices. They come from a separately instrumented binary whose per-launch event recording inflates the window itself by 8.4% (GB10) and 15.4% (RTX 4090) relative to the production binary, so we report them as proportions rather than absolute times. The production `h2d_ms` counter is deliberately omitted: it overlaps `gpu_l8_ms` and is not a transfer measurement (see text).

served largely from cache, whereas the GB10 must stream it from unified memory at close to hardware peak. We offer this as the reading most consistent with the measured bandwidths rather than a profiled fact, having not captured L2 hit-rate counters (Section VII).

Second, device-to-host transfer runs the other way: 3.46 ms on the GB10 against 16.42 ms on the RTX 4090, a 4.74× advantage for unified memory on identical payloads (35.1 vs. 7.4 GB/s effective). This is the cleanest unified-versus-discrete contrast in our data, and unlike the retracted H2D figure it is measured outside the event window, after `cudaEventSynchronize`, with no kernel execution to conflate it with.

Peak device memory stayed bounded at 45,382 KiB (≈ 44.3 MiB) on both platforms across every configuration, about 1 MiB above the 43.3 MiB theoretical minimum (Section II-C).

V. DISCUSSION AND KEY INSIGHTS

A. Speedup vs. Maxed-Out CPU Cores, and Warp Alignment

We report GPU speedup against CPU V1 rather than V0 at multi-thread configurations: V1 is race-free by construction at every thread count, while V0 is only *incidentally* correct at 1 thread and degrades thereafter. V0 is in fact the raw wall-time winner at 20 threads (6,346.33 ms, which would give only 1.33×), but its output there is not verified correct; we state the number rather than omit it.

GPU V2 (32×8_256) reaches 4,756.33 ms (31.5 FPS), a 1.46× speedup over CPU V1's 20-thread configuration (6,921.83 ms, the fastest correctness-verified CPU result). This is our headline figure, consistent with how we report the RTX 4090.

The serial comparison needs more care, because the two variants do not share a serial baseline. At 1 thread V0 is bit-exact and also faster than V1 (18,240.50 vs. 21,552.33 ms), since it skips the privatization V1 pays for, making it the fastest *bit-exact* serial baseline available. Measured against it, the serial figures are 3.84× (GB10) and 3.86× (RTX 4090). Against V1's serial time they rise to 4.53× and 4.21×; that keeps the denominator inside one variant family, but the slower denominator is what inflates the ratio, so we lead with 3.84× and report 4.53× only as the within-variant figure.

The 1.46× is against a non-trivial baseline: CNN inference scales well on ARM big.LITTLE clusters when properly scheduled [5], and across the full ladder the GB10's homogeneous-per-tier cores scale smoothly with none of the collapse the RTX 4090 shows past 8 threads (Section VI). GPU V2 is also the steadier configuration (±30.71 ms against CPU V1's ±82.49 ms at its own best setting).

Shifting from the default 16×16 grid (696.51 ± 1.78 ms) to the warp-aligned 32×8 grid (533.19 ± 2.58 ms) cuts Layer 8 kernel time by 163.32 ms (23.4%), a separation exceeding fifty standard deviations. We attribute it to memory coalescing: a 32-thread-wide block lets every thread in a warp issue into the same coalesced transaction, where a 16-wide block splits across several. The per-kernel breakdown tests this more sharply than wall time can. Block shape affects `deconv_kernel` only — `spatial_reduction_kernel` is launched one-dimensionally — so if coalescing is the mechanism, the whole gain must land in the deconvolution. It does: `deconv_kernel` drops 24.9% (564.74 to 424.07 ms) while the reduction moves +0.9% (26.08 to 26.32 ms), inside its own run-to-run spread. This remains an inference from timing rather than a direct load-efficiency measurement (Section VII).

B. Transfer Cost and Amdahl's Law

With the measurement error of Section IV-E corrected, transfer is not the bottleneck we took it for. On the GB10 everything inside the event window that is not kernel execution totals 127.65 ms, 2.7% of wall time, and D2H adds 3.46 ms; even charging that entire residual to transfer, which overstates it, moves under 3% of the pipeline. The RTX 4090's residual is larger both absolutely and as a share of its event window (152.14 ms, 36.5% against 22.1%), the direction PCIe versus NVLink-C2C predicts, though the margin is small enough that we do not lean on it. The one clean and clearly asymmetric

transfer measurement is D2H, where unified memory is $4.74\times$ faster.

What Table III shows instead is an Amdahl's Law [12] ceiling. Layer 8 is now 533.19 ms, 11.2% of GB10 wall time and 6.6% on the RTX 4090, while CPU Layers 1–7 — still on 20 and 4 OpenMP threads, not serial — account for 71.4% and 84.5%. The accelerated stage has been optimized into near-irrelevance: making Layer 8 free would still leave the RTX 4090 pipeline at 93% of current wall time. Offloading Layers 1–7 is therefore the only remaining path to sub-second reconstruction, and a Roofline analysis [14] would establish whether that offload is compute- or memory-bound first. The unaccounted remainder is process launch, file I/O and the per-frame `uint8`↔`double` conversion loops, all single-threaded and outside the instrumented region; it is larger on the GB10 (821.20 ms, 17.3%, against 469.33 ms, 8.6%), consistent with serial host code on a 3.9 GHz ARM core rather than a 6.0 GHz x86 one, though we did not instrument it to confirm.

VI. Cross-Platform Validation on Discrete-GPU Hardware

To test whether the GPU Spatial Reduction approach generalizes beyond the GB10's unified-memory architecture, we replicated the experiment on a conventional discrete-GPU desktop (Intel Core i9-14900K + NVIDIA RTX 4090; specifications in Table I), where CPU (system DDR) and GPU (VRAM) memory are physically separate and connected over PCIe rather than NVLink-C2C. Unlike the GB10's 20 homogeneous-per-tier ARM cores with no simultaneous multithreading (SMT), this CPU has a heterogeneous 8 Performance-core + 16 Efficiency-core layout where only the P-cores support Hyper-Threading, yielding 24 physical / 32 logical threads.

A. Methodology Adjustment and GPU Speedup Against a Fair Baseline

Reusing the GB10's thread ladder and 20-thread CPU backdrop produced misleading results here: wall time collapses past 8 threads, reaching $\sim 3.5\times$ slower than the 4-thread optimum, and that degraded configuration would have backdropped every GPU grid-search run. We therefore swept a topology-correct ladder (1, 2, 4, 8, 16, 24, 32) and used this platform's own best CPU configuration (4 threads) as the backdrop. The collapse reproduced across three separate full runs with tight SDs at each collapsed point (16 threads: $29{,}341 \pm 240$ ms, against $29{,}511 \pm 125$ and $29{,}609 \pm 249$ ms previously), so it is not an artifact. It is consistent with this chip's P/E-core split relying on Intel Thread Director while the generic OpenMP pool carries no affinity control [7] (`OMP_PLACES`/`proc_bind`), a known hazard for asymmetric-CPU system software [6]; but the 4-thread optimum does not cleanly match the 8 P-cores, so we do not claim a confirmed topological explanation and cannot exclude bandwidth or thermal effects without a control experiment (Section VII). The pathology is incidental to our contribution; we report it because it changes what a fair CPU baseline is on this hardware.

With the backdrop corrected, GPU V2 reaches $5{,}435$–$5{,}579$ ms (26.9–27.6 FPS), a $1.59\times$–$1.63\times$ speedup over CPU V1's best configuration (4 threads, $8{,}844.17$ ms), not the inflated $4\times$+ figure obtained by comparing against V1's serial time or its degraded 32-thread configuration. Under the same "best deterministic CPU" standard used on the GB10 (Section V-A), this exceeds that platform's $1.46\times$, consistent with V1's parallel scaling here being the more constrained of the two.

B. Cross-Architecture Bit-Exactness

Section IV-A establishes bit-exactness *within* each platform. A stronger question is whether GPU V2's output is identical *across* them. The GB10 (ARM host, Blackwell `sm_121`) and the RTX 4090 (x86 host, Ada Lovelace `sm_89`) differ in host ISA and GPU microarchitecture, so bit-identical output is not guaranteed by IEEE-754 `double` compliance alone: code generation for two SM targets, FMA contraction, and math-library implementations can each diverge in the last bit. The toolchains are otherwise closely matched (GCC 13.3.0 and CUDA 13.0 on both, differing only in patch level), so this isolates the ISA and microarchitecture difference rather than compounding it with a toolkit-version difference; a cross-*toolkit* claim would need a separate experiment. Running the winning 32×8 configuration on both and comparing raw outputs, `cmp -l` reports 0 differing bytes and the SHA-256 digests match (`8db2c07a...ca066b`). GPU V2's output is therefore bit-identical across two GPU generations and two host ISAs, the strongest form of the determinism claim this paper makes.

C. Warp Alignment Generalizes Across GPU Architectures

At the kernel level, 32×8 remains fastest on this Ada Lovelace GPU (360.92 ± 7.22 ms over 6 runs), 17.4% faster than 16×16 (437.06 ± 3.86 ms) and 17.5% faster than the slowest configuration (437.49 ± 3.46 ms), confirming the warp-aligned coalescing argument from Section III-B (a consequence of CUDA's 32-thread warp granularity [8]) is a property of CUDA itself, not specific to the GB10. The same localization holds here: `deconv_kernel` falls 15.6% (310.32 to 261.90 ms) while `spatial_reduction_kernel` is unchanged at 2.62 ms in both configurations, matching the GB10's pattern on a different microarchitecture. At the *wall-clock* level the five configurations remain statistically hard to distinguish ($5{,}435$–$5{,}579 \pm 101$–222 ms, overlapping error bars) even though 32×8 nominally wins both metrics here; the kernel-level advantage is only cleanly visible via CUDA-event profiling, reinforcing why Section IV-A's instrumentation matters. Figure 2 compares kernel time across all five configurations on both GPUs.

D. Non-Unified Memory and FP64 Precision

This platform's CPU and GPU memory are physically separate over PCIe, unlike the GB10's unified pool over NVLink-C2C [20], [21]. Despite that, GPU V2's Layer 8 kernel (360.92 ± 7.22 ms) is faster in absolute terms than the

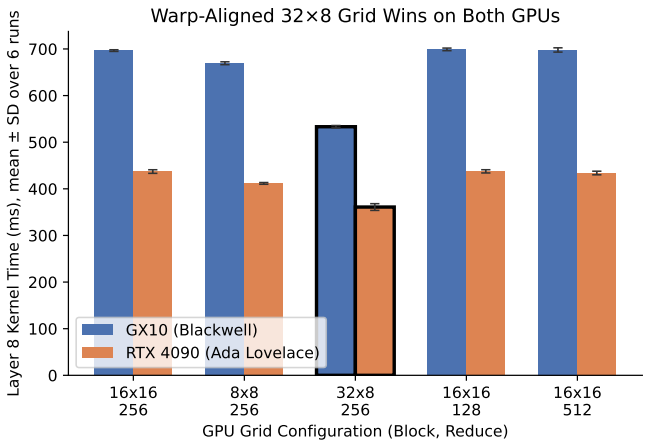

Fig. 2. GPU Layer 8 kernel execution time by grid configuration, GB10 (Blackwell) vs. RTX 4090 (Ada Lovelace). The warp-aligned 32×8 grid (outlined) is fastest on both GPU architectures.

GB10's (533.19 ± 2.58 ms), and Table III shows the answer is not a single effect. Discrete-GPU PCIe/DMA overhead [9] is visible — in-window residual is 36.5% of this GPU's event window against 22.1%, and D2H is $4.74\times$ slower — but it is outweighed twice over: the deconvolution runs $1.6\times$ faster and the bandwidth-bound reduction $10.0\times$ faster, the latter because the private buffer fits this card's L2. The discrete-memory penalty is therefore real and measurable rather than absent; it is simply smaller than the compute and cache advantages at this payload size, and we would expect the balance to shift once the working set overflows that L2.

Double precision (Section IV-A) has a real cost here: the RTX 4090's FP64 throughput is architecturally $1/64$ of its FP32 ($\approx$ 1.3 vs. $\approx$ 82.6 TFLOPS), so this kernel time is achieved with the card's dominant compute resource largely idle. An FP32 implementation would plausibly be substantially faster, at the price of weakening bit-exactness to a numerical-tolerance claim. The GB10's datacenter-class GPU has a much smaller FP64:FP32 gap, so the trade-off is less severe there (not separately quantified).

VII. Limitations and Future Work

The largest gap is the absence of a competitive GPU baseline: V2 is compared only against CPU implementations, not against `atomicAdd` accumulation, an output-stationary gather kernel, or cuDNN. Second, our explanation for the $10\times$ reduction-kernel gap — that the private buffer fits the RTX 4090's L2 but not the GB10's — rests on effective bandwidths, one of which exceeds DRAM peak; L2 hit-rate counters would confirm it directly, and a working-set sweep across the L2 boundary would test the design rule we draw from it. The coalescing explanation is likewise an inference from timing, though now localized to `deconv_kernel` as predicted; profiler load-efficiency counters and a sweep including non-aligned widths (e.g. 64×4) would settle it.

Three narrower gaps remain. After retracting the `h2d_ms` reading, our host-to-device figure is an upper bound still containing launch overhead and host-side padding; pinned-memory or multi-stream instrumentation would separate them, and the D2H measurement is unaffected. No FP32 ablation was run to quantify the speedup-versus-bit-exactness trade-off. And no `OMP_PROC_BIND`/`OMP_PLACES` control experiment isolates the RTX 4090 collapse from bandwidth saturation, thermal throttling, or false sharing.

Finally, all results use one 150-frame QCIF-to-CIF sequence. Reconstruction quality (Section IV-C) is measured at a single scale factor with a bicubic-downsampled stand-in for a natively high-resolution source, and chroma is replicated rather than reconstructed, which bounds the pipeline's perceptual quality independently of Layer 8. Higher resolutions would shift the compute-to-transfer ratio and, per the cache argument above, push the private buffer past the RTX 4090's L2 — the regime where we expect our design rule to be most testable.

VIII. Conclusion

We introduced a GPU spatial-reduction framework for FSRCNN's Layer 8 deconvolution and evaluated it on two architecturally distinct platforms. On both, the CPU and GPU spatial-reduction variants are bit-exact against a verified deterministic reference at every configuration tested, while naive accumulation corrupts output non-monotonically — as low as 13.8 dB on the GB10, and a near-undetectable 68.6 dB on the RTX 4090, which is the more instructive failure. GPU output is further bit-identical *across* the two platforms, spanning two host ISAs and two GPU generations. The warp-aligned 32×8 grid is fastest on both, and per-kernel profiling localizes the entire gain to the deconvolution kernel exactly as the coalescing explanation requires. Speedups over each platform's best correctness-verified CPU configuration are $1.46\times$ (31.5 FPS) and $1.63\times$, with $3.84\times$ against the fastest bit-exact serial run.

Per-kernel profiling also corrected an error in our own earlier instrumentation: what we had read as a large host-to-device transfer asymmetry was a host-side counter double-counting kernel-completion waiting, and we retract it. The genuine asymmetries run in opposite directions. Unified memory returns results $4.74\times$ faster, while the discrete GPU runs the bandwidth-bound reduction $10.0\times$ faster because its 72 MB L2 holds the entire 43.31 MiB private buffer that the GB10 must stream at 95% of memory peak. The cost of determinism thus depends less on the algorithm than on whether the privatization working set fits in last-level cache — a boundary that should transfer to any network where this pattern is applied.

References

[1] Annisa, Adnan, and Z. Zainuddin, "Enhancing computational efficiency: Transitioning from serial to parallel programming for low-to-high resolution video reconstruction using FSRCNN on AMP architecture," in *Proc. IEEE Int. Conf. Artificial Intelligence and Mechatronics Systems (AIMS)*, 2025, pp. 1–8, doi: 10.1109/AIMS66189.2025.11229650.

[2] C. Dong, C. C. Loy, and X. Tang, "Accelerating the Super-Resolution Convolutional Neural Network," in *Proc. European Conference on Computer Vision (ECCV)*, 2016, pp. 391–407.

[3] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[4] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738–1762, 2019.

[5] S. Wang, G. Ananthanarayanan, Y. Zeng, N. Goel, A. Pathania, and T. Mitra, "High-Throughput CNN Inference on Embedded ARM big.LITTLE Multicore Processors," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 10, pp. 2254–2267, Oct. 2020.

[6] C. Bilbao, J. C. Saez, and M. Prieto-Matías, "Flexible system software scheduling for asymmetric multicore systems with PMCSched: A case for Intel Alder Lake," *Concurrency and Computation: Practice and Experience*, vol. 35, no. 25, e7814, 2023, doi: 10.1002/cpe.7814.

[7] A. E. Eichenberger, C. Terboven, M. Wong, and D. an Mey, "The Design of OpenMP Thread Affinity," in *OpenMP in a Heterogeneous World (IWOMP 2012)*, Lecture Notes in Computer Science, vol. 7312, Springer, 2012, pp. 15–28, doi: 10.1007/978-3-642-30961-8_2.

[8] NVIDIA Corporation, *CUDA C++ Programming Guide*, [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/.

[9] Y. Go, M. A. Jamshed, Y. Moon, C. Hwang, and K. Park, "APUNet: Revitalizing GPU as Packet Processing Accelerator," in *Proc. 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2017, pp. 83–96.

[10] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, Apr. 2004.

[11] C. Dong, C. C. Loy, K. He, and X. Tang, "Learning a Deep Convolutional Network for Image Super-Resolution," in *Proc. European Conference on Computer Vision (ECCV)*, 2014, pp. 184–199, doi: 10.1007/978-3-319-10593-2_13.

[12] G. M. Amdahl, "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities," in *Proc. AFIPS Spring Joint Computer Conference*, 1967, pp. 483–485, doi: 10.1145/1465482.1465560.

[13] K. Serebryany and T. Iskhodzhanov, "ThreadSanitizer: Data Race Detection in Practice," in *Proc. Workshop on Binary Instrumentation and Applications (WBIA)*, 2009, pp. 62–71, doi: 10.1145/1791194.1791203.

[14] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009, doi: 10.1145/1498765.1498785.

[15] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, "cuDNN: Efficient Primitives for Deep Learning," arXiv:1410.0759, 2014.

[16] V. Dumoulin and F. Visin, "A Guide to Convolution Arithmetic for Deep Learning," arXiv:1603.07285, 2016.

[17] NVIDIA Corporation, "NVIDIA Puts Grace Blackwell on Every Desk and at Every AI Developer's Fingertips," NVIDIA Newsroom, Jan. 2025. [Online]. Available: https://nvidianews.nvidia.com/news/nvidia-puts-grace-blackwell-on-every-desk-and-at-every-ai-developers-fingertips.

[18] Arm Limited, "DynamIQ Technology." [Online]. Available: https://www.arm.com/technologies/dynamiq.

[19] NVIDIA Corporation, *NVIDIA Ada GPU Architecture*, Whitepaper V2.02, 2022. [Online]. Available: https://images.nvidia.com/aem-dam/Solutions/geforce/ada/nvidia-ada-gpu-architecture.pdf.

[20] NVIDIA Corporation, "NVIDIA Grace Hopper Superchip Architecture In-Depth," NVIDIA Developer Technical Blog, Nov. 2022. [Online]. Available: https://developer.nvidia.com/blog/nvidia-grace-hopper-superchip-architecture-in-depth/.

[21] J. Wahlgren, G. Schieffer, R. Shi, E. A. León, R. Pearce, M. Gokhale, and I. Peng, "Dissecting CPU-GPU Unified Physical Memory on AMD MI300A APUs," in *Proc. IEEE Int. Symp. Workload Characterization (IISWC)*, 2025, doi: 10.1109/IISWC66894.2025.00038.

[22] A. Lashgar, A. Baniasadi, and A. Khonsari, "Dynamic Warp Resizing in High-Performance SIMT," arXiv:1208.2374, 2012, doi: 10.48550/arXiv.1208.2374.